

## Assignment 4 — Math Fundamentals for Robotics 16-811, Fall 2024

Mukai Yu  
MSR, CMU  
Pittsburgh, PA  
mukaiy@andrew.cmu.edu

1. Consider the following relationship between a function  $y(x)$  and its derivative  $y'(x)$  over the interval  $[0, 1]$ :

$$3y^2y' = 1, \text{ with } y(1) = 1.$$

(Caution: The “initial condition” is specified at the right endpoint in this problem.)

- (a) By hand, obtain an exact analytic solution  $y(x)$  to this differential equation.  
(You may simply guess  $y(x)$  and then show that it satisfies the differential equation.)

**Solution:**

We can always try Laplace transform but here's a simpler method:

$$\begin{aligned} 3y^2y' &= 3y^2 \frac{dy}{dx} = 1 \\ 3y^2 dy &= dx \\ \int 3y^2 dy &= \int dx \\ y^3 &= x + C \end{aligned}$$

Plug in  $y(1) = 1$ :

$$\begin{aligned} 1^3 &= 1 + C \\ C &= 0 \end{aligned}$$

So the solution is  $y(x) = x^{\frac{1}{3}}$ .

Verification:

$$3y^2y' = 3x^{\frac{2}{3}} \frac{1}{3}x^{-\frac{2}{3}} = 1$$

In the next three parts you will solve the differential equation numerically. You may **not** assume that you know the solution from part (a) within your solvers. Use part (a) merely to compute errors between your numerical solutions and the exact solution.

- (b) Implement and use Euler's method to solve the differential equation numerically over the interval  $[0, 1]$ . Use a step size of 0.05. How accurate is your numerical solution? (Compare the numerical solution to the exact solution, perhaps as follows: Create a table with one row for each  $x_i$  encountered using the given step size. The row might mention  $x_i$ , the true value  $y(x_i)$ , the estimated value  $y_i$  computed using Euler's method, and the error  $y(x_i) - y_i$ . Graph the results as well. — You are about to implement some additional methods. You may wish to combine all the methods of this problem into one big table along with one composite graph.)
- (c) Implement and use a fourth-order Runge-Kutta method to solve the differential equation numerically. Again, use a step size of 0.05. Again, how accurate is your numerical solution?
- (d) Finally, implement and use fourth-order Adams-Bashforth for the differential equation. Again, use a step size of 0.05. Initialize the iteration with the following four values:

$$\begin{aligned} y(1.15) &= 1.04768955317165, \\ y(1.10) &= 1.03228011545637, \\ y(1.05) &= 1.01639635681485, \\ y(1) &= 1. \end{aligned}$$

Once again, how accurate is your numerical solution?

**Solution:**

Combined Code:

```
# combine tables together

h = -0.05
a, b = 1, 0
N = int(abs((a - b) / h))

# Euler
y_euler = np.zeros(N + 1)
y_euler[0] = 1

# Runge-Kutta
y_rk = np.zeros(N + 1)
y_rk[0] = 1

# Adams
y_adams = np.zeros(N + 4)
y_adams[3] = 1
y_adams[2] = 1.01639635681485
y_adams[1] = 1.03228011545637
y_adams[0] = 1.04768955317165

# y' = 1 / (3y^2)

for n in range(N):
    x = a + n * h

    # Euler
    y_euler_prime = 1 / (3 * y_euler[n] ** 2)
    y_euler[n + 1] = y_euler[n] + h * y_euler_prime

    # Runge-Kutta
    k = np.zeros(4)
    k[0] = 1 / (3 * y_rk[n] ** 2)
    k[1] = 1 / (3 * (y_rk[n] + h * k[0] / 2) ** 2)
    k[2] = 1 / (3 * (y_rk[n] + h * k[1] / 2) ** 2)
    k[3] = 1 / (3 * (y_rk[n] + h * k[2]) ** 2)
    y_rk[n + 1] = y_rk[n] + h / 6 * (k[0] + 2 * k[1] + 2 * k[2] + k[3])

    # Adams
    f = np.zeros(4)
    previous_y = y_adams[n : n + 4]
    f = 1 / (3 * previous_y**2)
    y_adams[n + 4] = y_adams[n + 3] + h / 24 * (
        55 * f[-1] - 59 * f[-2] + 37 * f[-3] - 9 * f[-4]
    )

    print(
        f"n = {n}, x_{n} = {x}, y(x_{n}) = {x ** (1/3)}, y_euler_{n} = {y_euler[n]}, y(
            x_{n}) - y_euler_{n} = {x ** (1/3) - y_euler[n]}, y_runge_{n} = {y_rk[n]}, y
            (x_{n}) - y_runge_{n} = {x ** (1/3) - y_rk[n]}, y_adams_{n} = {y_adams[n +
            3]}, y(x_{n}) - y_adams_{n} = {x ** (1/3) - y_adams[n + 3]}"
    )

print(
    f"n = {N}, x_{N} = {b}, y(x_{N}) = {b ** (1/3)}, y_euler_{N} = {y_euler[N]}, y(x_{N})
    - y_euler_{N} = {b ** (1/3) - y_euler[N]}, y_runge_{N} = {y_rk[N]}, y(x_{N}) -
    y_runge_{N} = {b ** (1/3) - y_rk[N]}, y_adams_{N} = {y_adams[N + 3]}, y(x_{N}) -
    y_adams_{N} = {b ** (1/3) - y_adams[N + 3]}"
)
```

Combined Table:

| n  | $x_i$  | True $y(x_i)$ | Euler's Method |                | Runge-Kutta |                | Adams-Bashforth |                |
|----|--------|---------------|----------------|----------------|-------------|----------------|-----------------|----------------|
|    |        |               | $y_i$          | $y(x_i) - y_i$ | $y_i$       | $y(x_i) - y_i$ | $y_i$           | $y(x_i) - y_i$ |
| 0  | 1.0    | 1.0           | 1.0            | 0.0            | 1.0         | 0.0            | 1.0             | 0.0            |
| 1  | 0.95   | 0.9830        | 0.9833         | -0.0002        | 0.9830      | 2.3154e-10     | 0.9830          | -3.1326e-07    |
| 2  | 0.9    | 0.9654        | 0.9660         | -0.0006        | 0.9654      | 5.3542e-10     | 0.9654          | -7.3195e-07    |
| 3  | 0.85   | 0.9472        | 0.9482         | -0.0009        | 0.9472      | 9.3811e-10     | 0.9472          | -1.2671e-06    |
| 4  | 0.8    | 0.9283        | 0.9297         | -0.0013        | 0.9283      | 1.4780e-09     | 0.9283          | -1.9753e-06    |
| 5  | 0.75   | 0.9085        | 0.9104         | -0.0018        | 0.9085      | 2.2119e-09     | 0.9085          | -2.9209e-06    |
| 6  | 0.7    | 0.8879        | 0.8903         | -0.0024        | 0.8879      | 3.2260e-09     | 0.8879          | -4.2000e-06    |
| 7  | 0.6499 | 0.8662        | 0.8692         | -0.0030        | 0.8662      | 4.6544e-09     | 0.8662          | -5.9595e-06    |
| 8  | 0.6    | 0.8434        | 0.8472         | -0.0037        | 0.8434      | 6.7129e-09     | 0.8434          | -8.4267e-06    |
| 9  | 0.55   | 0.8193        | 0.8240         | -0.0046        | 0.8193      | 9.7604e-09     | 0.8193          | -1.1965e-05    |
| 10 | 0.5    | 0.7937        | 0.7994         | -0.0057        | 0.7937      | 1.4421e-08     | 0.7937          | -1.7181e-05    |
| 11 | 0.4499 | 0.7663        | 0.7733         | -0.0070        | 0.7663      | 2.1834e-08     | 0.7663          | -2.5118e-05    |
| 12 | 0.3999 | 0.7368        | 0.7455         | -0.0087        | 0.7368      | 3.4209e-08     | 0.7368          | -3.7679e-05    |
| 13 | 0.35   | 0.7047        | 0.7155         | -0.0108        | 0.7047      | 5.6160e-08     | 0.7047          | -5.8534e-05    |
| 14 | 0.2999 | 0.6694        | 0.6829         | -0.0135        | 0.6694      | 9.8257e-08     | 0.6695          | -9.5318e-05    |
| 15 | 0.25   | 0.6299        | 0.6472         | -0.0172        | 0.6299      | 1.8783e-07     | 0.6301          | -0.0001        |
| 16 | 0.1999 | 0.5848        | 0.6074         | -0.0226        | 0.5848      | 4.0811e-07     | 0.5851          | -0.0003        |
| 17 | 0.1499 | 0.5313        | 0.5623         | -0.0309        | 0.5313      | 1.0798e-06     | 0.5320          | -0.0006        |
| 18 | 0.0999 | 0.4641        | 0.5096         | -0.0454        | 0.4641      | 3.9945e-06     | 0.4659          | -0.0018        |
| 19 | 0.0499 | 0.3684        | 0.4454         | -0.0770        | 0.3683      | 2.8737e-05     | 0.3755          | -0.0071        |
| 20 | 0      | 0.0           | 0.3614         | -0.3614        | 0.1034      | -0.1034        | 0.2208          | -0.2208        |

TABLE I: Comparison of Numerical Methods

Solution Accuracy:

$$\begin{cases} \text{Euler's Method:} & |e| \approx 0.3614 \\ \text{Runge-Kutta Method:} & |e| \approx 0.1034 \\ \text{Adams-Bashforth Method:} & |e| \approx 0.2208 \end{cases} \quad (1)$$

Looks like Adams-Bashforth is not always better than Runge-Kutta.

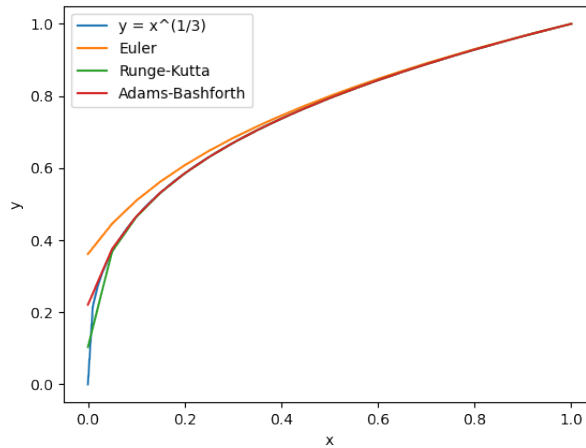


Fig. 1: Comparison of Numerical Methods

Please submit code for parts (b)-(d). Include in your pdf any relevant code snippets that you would like the TAs to see, along with your table and graph, plus your work for (a).

2. Consider the function  $f(x, y) = x^3 + y^3 - 2x^2 + 3y^2 - 8$  (for real  $x$  and  $y$ ).

- (a) Find all critical points of  $f$  by sketching in the  $(x, y)$ -plane the iso-contours  $\frac{\partial f}{\partial x} = 0$  and  $\frac{\partial f}{\partial y} = 0$ . Then classify the critical points into local minima, local maxima, and saddle points by considering either nearby gradient directions or the Hessian of  $f$ .

**Solution:**

$$\begin{aligned}\frac{\partial f}{\partial x} &= 3x^2 - 4x = 0 \Rightarrow x = 0, \frac{4}{3} \\ \frac{\partial f}{\partial y} &= 3y^2 + 6y = 0 \Rightarrow y = 0, -2\end{aligned}$$

Plug into the original function we can get

$$\begin{cases} f(0, y) &= y^3 + 3y^2 - 8 \\ f(\frac{4}{3}, y) &= \frac{64}{27} + y^3 - \frac{32}{9} + 3y^2 - 8 \\ f(x, 0) &= x^3 - 2x^2 - 8 \\ f(x, -2) &= x^3 - 2x^2 - 4 \end{cases}$$

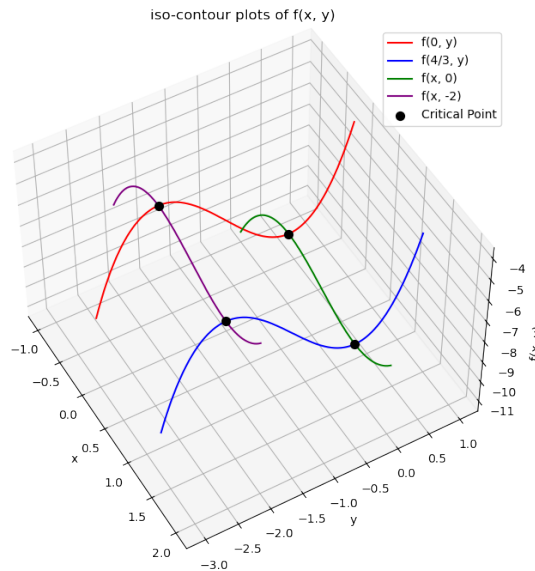


Fig. 2: iso-contours of  $f$

The critical points are:

$$\begin{cases} (0, 0) \\ (0, -2) \\ (\frac{4}{3}, 0) \\ (\frac{4}{3}, -2) \end{cases}$$

Gradient vector and Hessian matrix of  $f$ :

$$\begin{cases} \nabla f(x, y) = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix} = \begin{bmatrix} 3x^2 - 4x \\ 3y^2 + 6y \end{bmatrix} \\ H_f(x, y) = \begin{bmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y^2} \end{bmatrix} = \begin{bmatrix} 6x - 4 & 0 \\ 0 & 6y + 6 \end{bmatrix} \end{cases}$$

Hessian at each critical point:

$$\begin{cases} H_f(0,0) = \begin{bmatrix} -4 & 0 \\ 0 & 6 \end{bmatrix}, \det(H_f(0,0)) < 0 \Rightarrow H_f(0,0) \text{ indefinite} \\ H_f(0,-2) = \begin{bmatrix} -4 & 0 \\ 0 & -6 \end{bmatrix}, \det(H_f(0,-2)) > 0 \Rightarrow H_f(0,-2) \text{ negative definite} \\ H_f(\frac{4}{3},0) = \begin{bmatrix} 4 & 0 \\ 0 & 6 \end{bmatrix}, \det(H_f(\frac{4}{3},0)) > 0 \Rightarrow H_f(\frac{4}{3},0) \text{ positive definite} \\ H_f(\frac{4}{3},-2) = \begin{bmatrix} 4 & 0 \\ 0 & -6 \end{bmatrix}, \det(H_f(\frac{4}{3},-2)) < 0 \Rightarrow H_f(\frac{4}{3},-2) \text{ indefinite} \end{cases}$$

So we can determine the type of critical points

$$\begin{cases} (0,0) & \text{saddle point} \\ (0,-2) & \text{local maximum} \\ (\frac{4}{3},0) & \text{local minimum} \\ (\frac{4}{3},-2) & \text{saddle point} \end{cases}$$

- (b) By hand, show how steepest descent would behave starting from the point  $(x,y) = (1,-1)$ . Use the version of steepest descent that moves to the nearest local minimum on the negative gradient line, as discussed in class. How many such steepest descent steps are needed to converge to an overall local minimum of  $f$ ?

**Solution:**

The gradient of  $f$  at  $(1,-1)$  is  $\nabla f(1,-1) = \begin{bmatrix} -1 \\ -3 \end{bmatrix}$ , so the direction is along the negative gradient vector  $-\nabla f(1,-1) = \begin{bmatrix} 1 \\ 3 \end{bmatrix}$ .

So we can express the line as:

$$\begin{cases} x = 1 + t \\ y = -1 + 3t \end{cases}$$

And perform a line minimization by substituting  $x = 1 + t$  and  $y = -1 + 3t$  into  $f(x,y)$ :

$$f(t) = (1+t)^3 + (-1+3t)^3 - 2(1+t)^2 + 3(-1+3t)^2 - 8$$

We can then find the minimum of  $f(t)$  by taking the derivative and setting it to zero:

$$\begin{aligned} \frac{df}{dt} &= 3(1+t)^2 + 3(-1+3t)^2 \cdot 3 - 4(1+t) + 6(-1+3t) \cdot 3 \\ &= (3+81)t^2 + (6-54-4+54)t + (3+9-4-18) \\ &= 84t^2 + 2t - 10 \\ &= 2(3t-1)(14t+5) = 0 \end{aligned}$$

So the 2 roots are:

$$\begin{cases} t_1 = \frac{1}{3} \\ t_2 = -\frac{5}{14} \end{cases}$$

To determine which one is the minimum, we need to check the second derivative:

$$\frac{d^2}{dt^2} f(t) = 168t + 2$$

Plug in the 2 roots we get:

$$\begin{cases} \frac{d^2}{dt^2} f(\frac{1}{3}) &= 168 \cdot \frac{1}{3} + 2 = 60 > 0 \Rightarrow \text{local minimum} \\ \frac{d^2}{dt^2} f(-\frac{5}{14}) &= 168 \cdot (-\frac{5}{14}) + 2 = -50 < 0 \Rightarrow \text{local maximum} \end{cases}$$

Plug in the minimum point  $t = \frac{1}{3}$  back we get:

$$\begin{cases} x = 1 + \frac{1}{3} = \frac{4}{3} \\ y = -1 + 3 \cdot \frac{1}{3} = 0 \end{cases}$$

Which corresponds to the local minimum we get in part (a).

With just 1 steepest descent step, we can converge to the local minimum. Literally zero-shot local minimum!

Please explain how you got your results and show your work. No code is needed or expected.

3. Let  $Q$  be a real symmetric positive definite  $n \times n$  matrix. Show that any two eigenvectors of  $Q$  corresponding to distinct eigenvalues of  $Q$  are  $Q$ -orthogonal. Show this directly from the definition of eigenvector, without assuming eigenvectors are orthogonal in the usual sense.

**Solution:**

Let  $v_1$  and  $v_2$  be eigenvectors of  $Q$  corresponding to distinct eigenvalues  $\lambda_1$  and  $\lambda_2$  respectively,  $\lambda_1 \neq \lambda_2$ . Then we have:

$$Qv_1 = \lambda_1 v_1$$

$$Qv_2 = \lambda_2 v_2$$

Then we can derive the 2 forms of  $v_1^T Q v_2$ :

$$v_1^T Q v_2 = v_1^T \lambda_2 v_2 = \lambda_2 v_1^T v_2 \quad (2)$$

$$v_1^T Q v_2 = v_1^T Q^T v_2 = (Qv_1)^T v_2 = \lambda_1 v_1^T v_2 \quad (3)$$

Subtract (2) from (3) we get:

$$(\lambda_2 - \lambda_1) v_1^T v_2 = 0$$

Because  $\lambda_1 \neq \lambda_2$ , we have  $v_1^T v_2 = 0$ , the usual sense of orthogonal eigen vectors.

Now plug this back into (2) or (3) we get:

$$v_1^T Q v_2 = 0$$

So  $v_1$  and  $v_2$  are  $Q$ -orthogonal.

No code is needed or expected for this problem. Please put your work into the pdf.

- 4.(a) Show that in the purely quadratic form of the conjugate gradient method,  $d_k^T Q d_k = -d_k^T Q g_k$ . Consequently, show that to obtain  $x_{k+1}$  from  $x_k$  one does not need to use  $Q$  explicitly, assuming one already has available the vectors  $g_k$ ,  $Qg_k$ , and  $d_k$ .

(See the notes for notation:

- “Purely quadratic” means conjugate gradient applied to functions of the form  $f(x) = c + b^T x + \frac{1}{2} x^T Q x$ , with  $Q$  a real symmetric positive definite matrix.
- $g_k$  is the gradient of  $f(x)$  evaluated at  $x_k$ , and  $d_k$  is the descent direction along which the method moves from  $x_k$ .)

**Solution:**

Gradient and Hessian of  $f(x)$ :

$$\begin{cases} \nabla f(x) &= Qx + b \\ H_f(x) = \nabla^2 f(x) &= Q \end{cases}$$

And we have:

$$g_k = \nabla f(x_k) = Qx_k + b$$

Because it's steepest descent in each step, the descent direction  $d_k$  at  $x_k$  must be the negative gradient  $-g_k$ , so we have:

$$d_k = -g_k \Rightarrow d_k^T Q d_k = d_k^T Q (-g_k) = -d_k^T Q g_k$$

Then by the update step of conjugate gradient method we have:

$$\begin{aligned} x_{k+1} &= x_k + \alpha_k d_k \\ \alpha_k &= -\frac{g_k^T d_k}{d_k^T Q d_k} = \frac{g_k^T d_k}{d_k^T (Qg_k)} \end{aligned}$$

Thus we don't need to use  $Q$  explicitly to obtain  $x_{k+1}$ .

- (b) Show that in the purely quadratic form  $Qg_k$  can be computed by taking a unit step from  $x_k$  in the direction of the negative gradient and then evaluating the gradient there. Specifically, if  $y_k = x_k - g_k$  and  $p_k = \nabla f(y_k)$ , then  $Qg_k = g_k - p_k$ .

**Solution:**

$$p_k = \nabla f(y_k) = Qy_k + b = Q(x_k - g_k) + b = Qx_k - Qg_k + b = g_k - Qg_k \Rightarrow Qg_k = g_k - p_k$$

- (c) Combine the results of parts (a) and (b) to derive a conjugate gradient method for general functions  $f$  in the spirit of the algorithm presented in class, but which does not require knowledge of the Hessian of  $f$  or a line search. (In class we used line search to find minima of  $f$  along particular directions.) You may assume that the underlying ideas developed in (a) and (b) for purely quadratic functions carry over to general functions when you use the Conjugate Gradient Algorithm that appears on page 22 of the Optimization notes. Since that algorithm is based on the algorithm of page 17 in the notes, you will want to adapt the formulas for  $\alpha_k$  and  $\beta_k$  that appear on page 17.

**Solution:**

Combine the results of parts (a) and (b) we can derive:

$$\begin{aligned} \alpha_k &= \frac{g_k^T d_k}{d_k^T (Qg_k)} = \frac{g_k^T d_k}{d_k^T (g_k - p_k)} = \frac{g_k^T d_k}{d_k^T g_k - d_k^T p_k} \\ \beta_k &= \frac{g_{k+1}^T (Qd_k)}{d_k^T (Qd_k)} = \frac{g_{k+1}^T (g_k - p_k)}{d_k^T (g_k - p_k)} \end{aligned}$$

Updated version of Conjugate Gradient Algorithm for pure quadratic function  $f$  on page 17:

1. Let  $d_0 = -g_0 = -b_0 - Qx_0$  for randomly initialized  $x_0$ .
2. For  $k = 0, \dots, n-1$  do:
  3. a.  $\alpha_k \leftarrow \frac{g_k^T d_k}{d_k^T g_k - d_k^T p_k}$ , where  $p_k = \nabla f(x_k - g_k)$ .
  - b.  $x_{k+1} \leftarrow x_k + \alpha_k d_k$ .
  - c.  $\beta_k \leftarrow \frac{g_{k+1}^T (g_k - p_k)}{d_k^T (g_k - p_k)}$ , where  $g_{k+1} = \nabla f(x_{k+1})$ .
  - d.  $d_{k+1} \leftarrow -g_{k+1} + \beta_k d_k$ .
4. Return  $x_n$ .

Updated version of Conjugate Gradient Algorithm on page 22 assuming the underlying ideas developed in (a) and (b) for purely quadratic functions carry over to general functions  $f$ :

1. Randomly initialize  $x_0$ , set  $d_0 = -\nabla f(x_0)$ .
2. For  $k = 0, \dots, n - 1$  do:
3.   a. Set  $x_{k+1} = x_k + \alpha_k d_k$ , where  $\alpha_k = \frac{g_k^T d_k}{d_k^T g_k - d_k^T p_k}$  and  $p_k = \nabla f(x_k - g_k)$ .
- b. Set  $d_{k+1} = -\nabla f(x_{k+1}) + \beta_k d_k$ , where  $\beta_k = \frac{|\nabla f(x_{k+1})|^2}{|\nabla f(x_k)|^2}$ .
4. Return  $x_n$ .

No code is needed or expected for this problem. Please answer all parts in the pdf.

5. Find the rectangle of a given perimeter that has the greatest area, by solving the Lagrange multiplier first-order necessary conditions. Verify the second-order sufficiency conditions.

No code is needed or expected for this problem. Please put your work into the pdf.

**Solution:**

Let  $x$  and  $y$  be the sides of the rectangle, so the area is  $A = xy$  and the perimeter is  $P = 2x + 2y$ .

So formally, the problem is:

$$\begin{aligned} \max_{x,y} \quad & xy \\ \text{s.t.} \quad & x + y = \frac{P}{2} \end{aligned}$$

Let  $\mathcal{L}(x, y, \lambda) = xy - \lambda(x + y - \frac{P}{2})$ .

Then taking first derivative of  $\mathcal{L}$  we have:

$$\begin{cases} \frac{\partial \mathcal{L}}{\partial x} = y - \lambda = 0 \\ \frac{\partial \mathcal{L}}{\partial y} = x - \lambda = 0 \\ \frac{\partial \mathcal{L}}{\partial \lambda} = -x - y + \frac{P}{2} = 0 \end{cases} \Rightarrow \begin{cases} x = y = \frac{P}{4} \\ \lambda = \frac{P}{4} \end{cases}$$

**This gives us the maximum area of  $\frac{P^2}{16}$ .**

To verify the second-order sufficiency conditions, we need to compute the Hessian of  $\mathcal{L}$ :

$$H_{\mathcal{L}}(x, y) = \begin{bmatrix} \frac{\partial^2 \mathcal{L}}{\partial x^2} & \frac{\partial^2 \mathcal{L}}{\partial x \partial y} \\ \frac{\partial^2 \mathcal{L}}{\partial y \partial x} & \frac{\partial^2 \mathcal{L}}{\partial y^2} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

And we need to determine the feasible set vector of  $(x, y)$ , i.e. the vectors  $d = \begin{bmatrix} d_x \\ d_y \end{bmatrix}$  that are orthogonal to the gradient of the constraints  $\nabla g(x, y) = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ .

$$d^T \nabla g(x, y) = [d_x \quad d_y] \begin{bmatrix} 1 \\ 1 \end{bmatrix} = d_x + d_y = 0 \Rightarrow d = \begin{bmatrix} t \\ -t \end{bmatrix}$$

Observe that:

$$d^T H_{\mathcal{L}}(x, y) d = [t \quad -t] \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} t \\ -t \end{bmatrix} = -2t^2 < 0$$

So the Hessian  $H_{\mathcal{L}}(x, y)$  is negative definite on the feasible set vector of  $(x, y)$ , and the solution is a maximum.

6. In this problem, you are going to use trajectory optimization to find obstacle-free paths for a robot. You will start with an obstacle cost world and a straight line path that blasts through the obstacles, as in Figure 3 and Figure 4. We are providing some starter code to make it easier - see the files `trajectory_optimization.{m|py}`. For your code, you mainly just need to add the actual optimization code to one of these files.

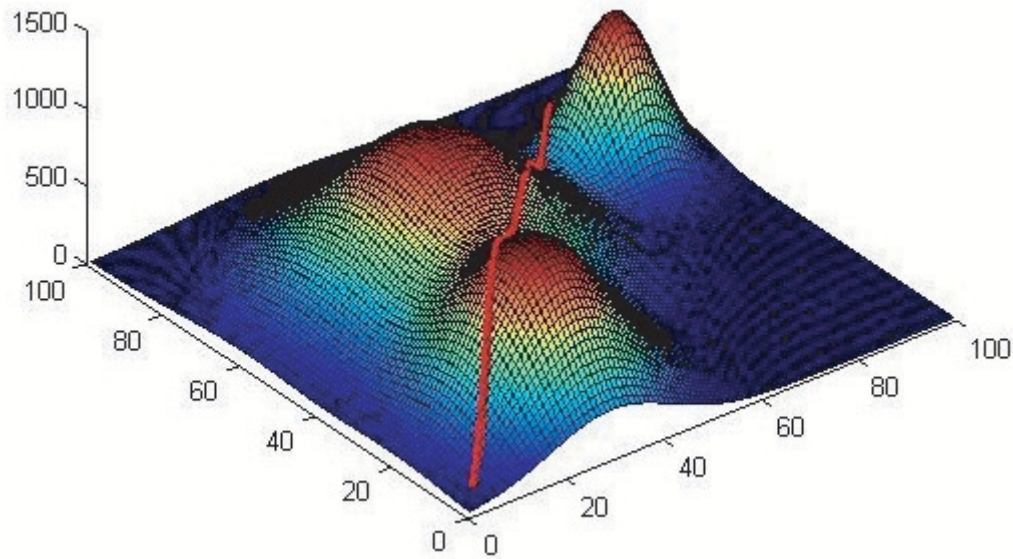


Fig. 3: A 2D world with obstacles depicted as cost functions. Shown in red is a straight line path between two locations. The path has high cost because it passes through/near obstacles.

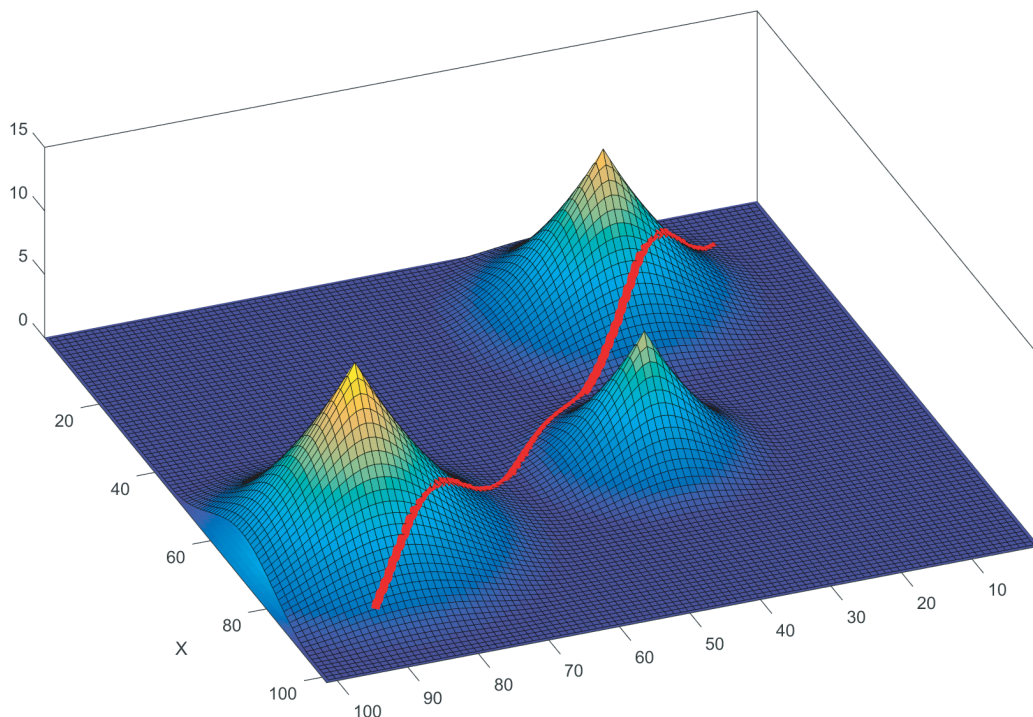


Fig. 4: Another 2D obstacle world and straight line path with moderately-high cost, corresponding to the code appearing in `trajectory_optimization.{m|py}`.

- (a) Starting from the straight line path that goes across the obstacle field as in Figure 4, perform gradient descent to find a path with locally optimal cost. You should represent the path as a sequence of 2D points, view that sequence as a

higher-dimensional point in some finite-dimensional space, and take gradient steps in that space. Rather than optimize along the entire gradient line as we did in lecture, simply scale the negative gradient by 0.1 and take a corresponding step.

Remember, this is a path for your robot, so make sure you don't accidentally optimize its start and goal as well; those should remain fixed.

Plot the path after one iteration — notice that it is moving away from the obstacles.

Pick convergence criteria and plot the path after convergence. What happened to it?

**Solution:**

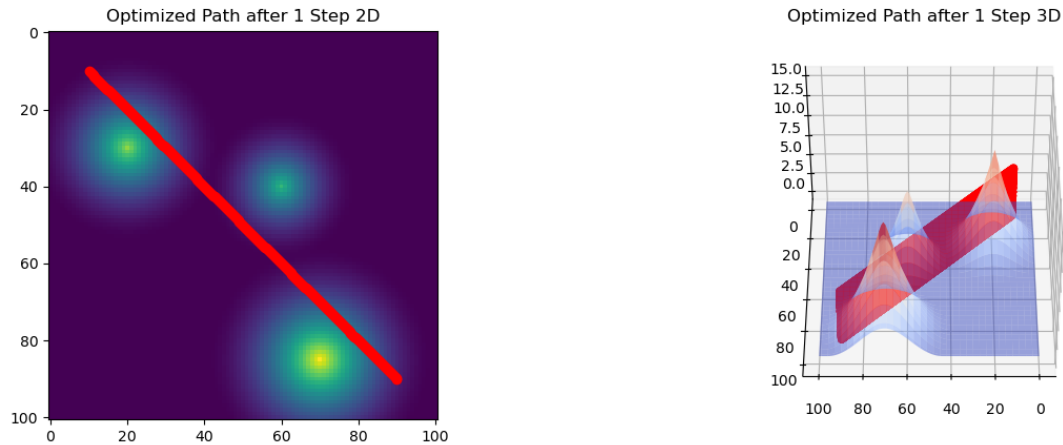


Fig. 5: Optimized Path After 1 Gradient Step for each Path Point.

I don't think the movement in 1 step is noticeable.

Now we set the convergence criterion to be the amount of cost change normalized by learning rate (step size), and the threshold is set to be  $10^{-3}$ .

```
# Now do continuous optimization with the amount of cost change as criterion
Qa_path_continuous = initial_path.copy()
optimization_points_continuous = Qa_path_continuous[1:-1]
convergence_map = np.zeros(optimization_points_continuous.shape[0], dtype=bool)
cost_interpolator = interpolate.RegularGridInterpolator((x, y), cost_map, bounds_error=False, fill_value=None)

cost_change_threshold = 1e-3
max_iter = 1000
lr = 0.1

for iter in range(max_iter):
    # if all points have converged, break
    if np.all(convergence_map):
        print(f"Converged after {iter} iterations")
        break

    for idx in range(len(optimization_points_continuous)):
        if convergence_map[idx]:
            continue

        point = optimization_points_continuous[idx]

        # Interpolate gradient at current point
        interpolated_gradient_x = gx_interpolator(point)
        interpolated_gradient_y = gy_interpolator(point)

        # Compute gradient vector
        gradient_vector = np.array(
            [interpolated_gradient_x, interpolated_gradient_y]
```

```

).flatten()

current_cost = cost_interpolator(point)

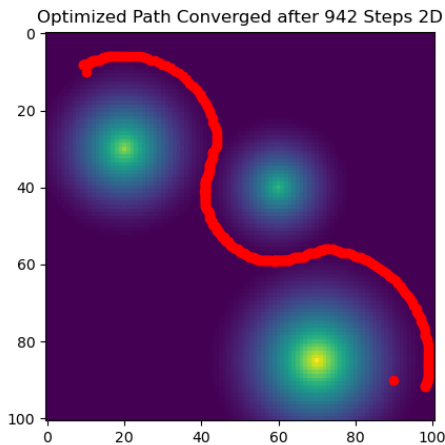
next_point = point - lr * gradient_vector
next_cost = cost_interpolator(next_point)

# Check if the point has converged
if (current_cost - next_cost) / lr < cost_change_threshold:
    convergence_map[idx] = True

# Take a step in the negative gradient direction
optimization_points_continuous[idx] = next_point

plot_scene(Qa_path_continuous, cost_map, title="Converged Optimized Path", save_prefix="
    ../figs/Q6_optimized_path_converged")

```



Optimized Path Converged after 942 Steps 3D

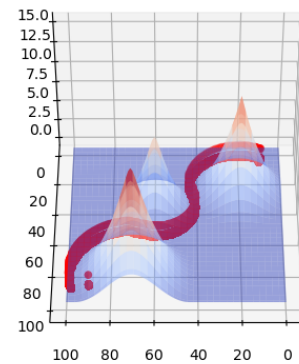


Fig. 6: Optimized Path Converged after 942 Iterations.

**It's not smooth at the start and end points, i.e., it does not respect the start and end point smoothness conditions.**

- (b) To avoid the issue from part (a), you need to tell the optimizer that this is a path instead of some independent 2D points.

One possibility is to consider an additional cost term,  $\min(\frac{1}{2}\|\xi_i - \xi_{i-1}\|^2)$ , that tells each 2D point  $\xi_i$  to be close to the previous 2D point  $\xi_{i-1}$  on the path. This additional term is sometimes called a *smoothness* cost.

Augment your negative gradient from part (a) with a new negative gradient, obtained for each  $\xi_i$  from the *single* term  $\frac{1}{2}\|\xi_i - \xi_{i-1}\|^2$ .

(The gradient of this new term with respect to  $\xi_i$  is  $(\xi_i - \xi_{i-1})$ . Include this term for  $\xi_i$ , but ignore this term when taking a gradient with respect to any other  $\xi_j$ , including  $\xi_{i-1}$ . This means we are really specifying a vector field for the optimizer to follow rather than an overall cost function. Another perspective is that each 2D point on the path is optimizing its own cost function, chosen in a convenient way.)

Weight the negative obstacle gradient from part (a) with 0.8 and the new negative gradient term with 4, add them together, then again scale that vector by 0.1 and take a corresponding step. (Weights like this are never set in stone; if you wish, you should feel free to experiment with other values.)

Plot the path after 100, 200 and 500 iterations. Why doesn't this work?

**Solution:**

Why do we need 2 weights for 1 single ratio? I just set the weight of obstacle cost to be 0.9 and smoothness to be 1 - obstacle cost.

```

for iter in range(max_iter):
    # if all points have converged, break
    if np.all(convergence_map):
        print(f"Converged after {iter} iterations")

```

```

        break

    if iter == 100:
        plot_scene(Qb_path_continuous, cost_map, title="Optimized Path after 100 Steps",
                    save_prefix="../figs/Q6b_optimized_path_100_steps")

    if iter == 200:
        plot_scene(Qb_path_continuous, cost_map, title="Optimized Path after 200 Steps",
                    save_prefix="../figs/Q6b_optimized_path_200_steps")

    if iter == 500:
        plot_scene(Qb_path_continuous, cost_map, title="Optimized Path after 500 Steps",
                    save_prefix="../figs/Q6b_optimized_path_500_steps")

for idx in range(len(optimization_points_continuous)):
    if convergence_map[idx]:
        continue

    point = optimization_points_continuous[idx]
    previous_path_point = Qb_path_continuous[idx] # using Qb_path_continuous for corner
        case when idx = 0

    # Interpolate gradient at current point
    interpolated_gradient_x = gx_interpolator(point)
    interpolated_gradient_y = gy_interpolator(point)

    # Compute gradient vector
    gradient_vector = np.array(
        [interpolated_gradient_x, interpolated_gradient_y]
    ).flatten()

    gradient_vector = obstacle_cost_weight * gradient_vector + (1 -
        obstacle_cost_weight) * (
        point - previous_path_point
    )

    current_cost = cost_interpolator(point)

    next_point = point - lr * gradient_vector
    next_cost = cost_interpolator(next_point)

    # Check if the point has converged
    if (current_cost - next_cost) / lr < cost_change_threshold:
        convergence_map[idx] = True

    # Take a step in the negative gradient direction
    optimization_points_continuous[idx] = next_point

```

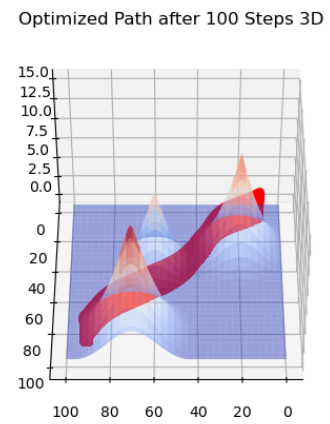
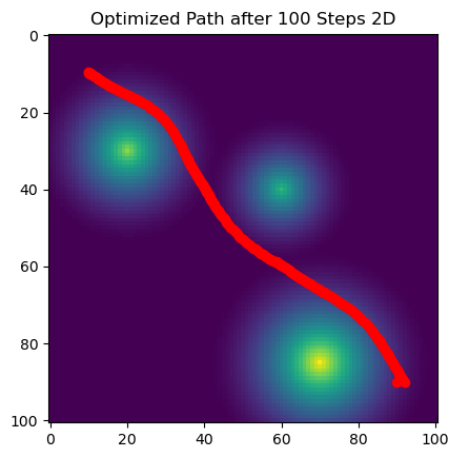


Fig. 7: Optimized Path after 100 Iterations.

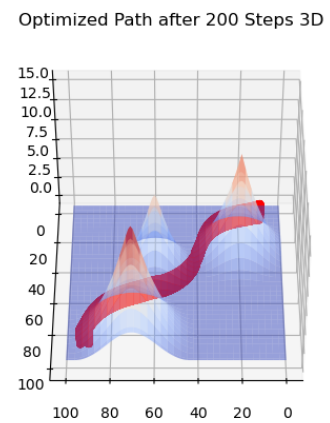
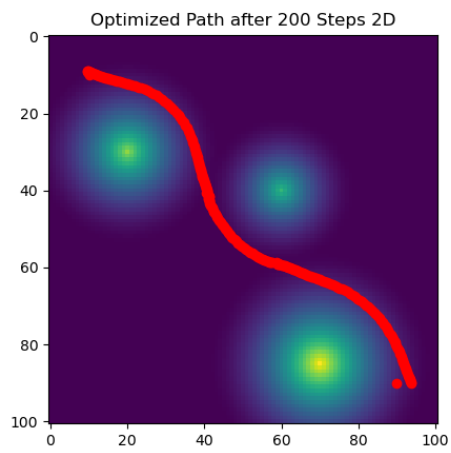


Fig. 8: Optimized Path after 200 Iterations.

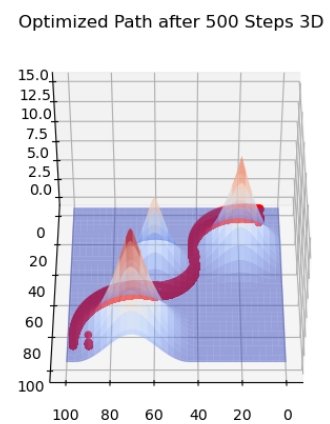
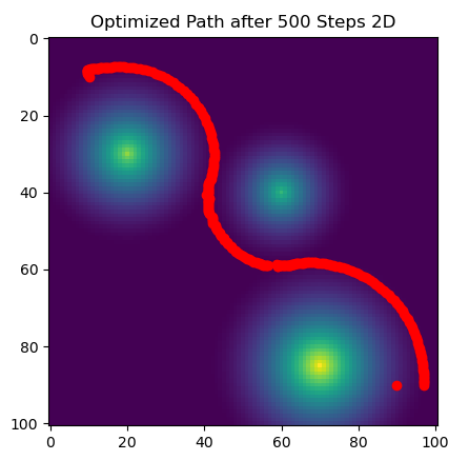
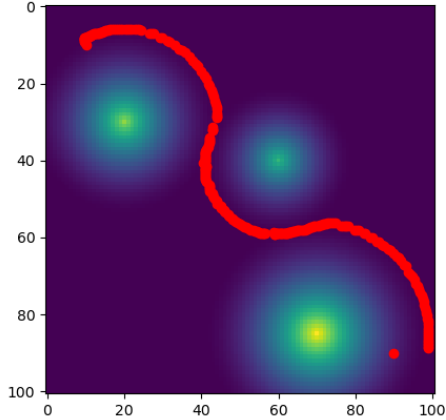


Fig. 9: Optimized Path after 500 Iterations.

Optimized Path with Previous Point Loss Converged after 1010 Steps 2D



Optimized Path with Previous Point Loss Converged after 1010 Steps 3D

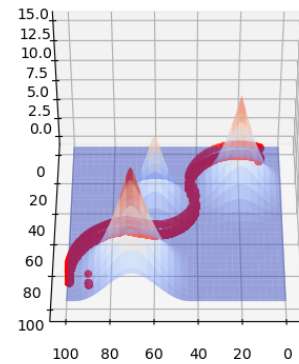


Fig. 10: Optimized Path Converged after 1010 Iterations.

**Does not respect end point smoothness condition.**

- (c) Finally, try augmenting the obstacle gradient with a different smoothness cost, telling each 2D point to be close to both its previous and next points on the path:

$$\min\left(\frac{1}{2}\|\xi_i - \xi_{i-1}\|^2 + \frac{1}{2}\|\xi_{i+1} - \xi_i\|^2\right).$$

Taking the gradient with respect to  $\xi_i$  yields  $(\xi_i - \xi_{i-1}) + (\xi_i - \xi_{i+1})$ , which is  $-\xi_{i-1} + 2\xi_i - \xi_{i+1}$ . (We can now view this gradient as derived from an overall cost function or we can again view it as derived for each 2D point from its own cost function.) Perform another optimization using this new smoothness cost to compute gradient updates:

- Use 0.8 to weight the obstacle cost and 4 to weight the smoothness cost. Then use a step-size scaling of 0.1 as before.
- Again, you should feel free to use different weights if your code works better with those, so long as the weights are not too different from those we suggested.

Plot the path after 100 and 5000 iterations. Why is this version better?

**Solution:**

```
for iter in range(max_iter):
    # if all points have converged, break
    if np.all(convergence_map) and converge_iter == 0:
        print(f"Converged after {iter} iterations")
        converge_iter = iter

    if iter == 100:
        plot_scene(Qc_path_continuous, cost_map, title="Optimized Path after 100 Steps",
                  save_prefix="../figs/Q6c_optimized_path_100_steps")

    for idx in range(len(optimization_points_continuous)):
        # if convergence_map[idx]:
        #     continue

        point = optimization_points_continuous[idx]
        previous_path_point = Qc_path_continuous[idx] # using Qc_path_continuous for corner
        case when idx = 0
        next_path_point = Qc_path_continuous[idx + 2] # using Qc_path_continuous for corner
        case when idx = the last index

        # Interpolate gradient at current point
        interpolated_gradient_x = gx_interpolator(point)
        interpolated_gradient_y = gy_interpolator(point)

        # Compute gradient vector
        gradient_vector = np.array(
```

```

        [interpolated_gradient_x, interpolated_gradient_y]
    ).flatten()

    gradient_vector = obstacle_cost_weight * gradient_vector + (1 -
        obstacle_cost_weight) * (
        -previous_path_point + 2 * point - next_path_point
    )

    current_cost = cost_interpolator(point)

    next_point = point - lr * gradient_vector
    next_cost = cost_interpolator(next_point)

    # Check if the point has converged
    if (current_cost - next_cost) / lr < cost_change_threshold:
        convergence_map[idx] = True

    # Take a step in the negative gradient direction
    optimization_points_continuous[idx] = next_point

```

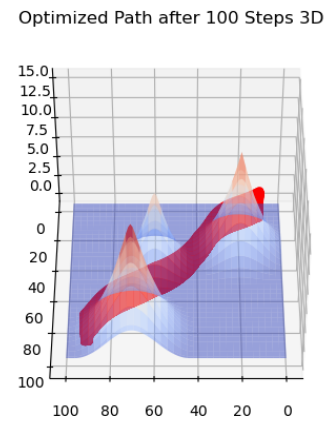
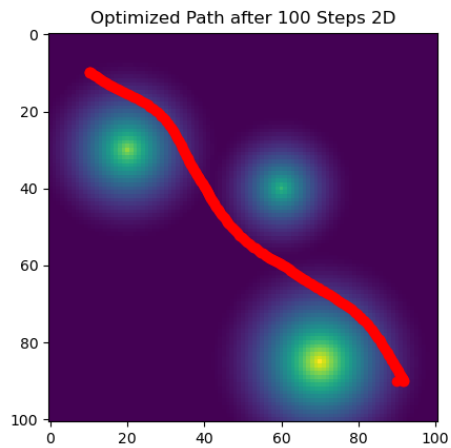


Fig. 11: Optimized Path after 100 Iterations.

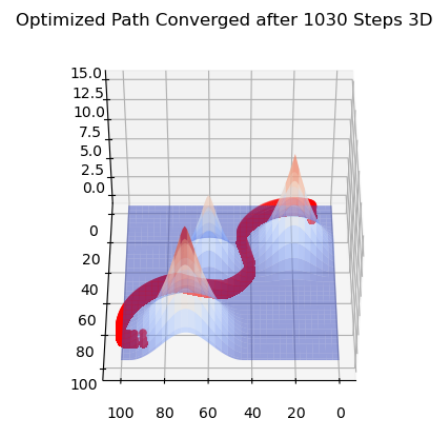
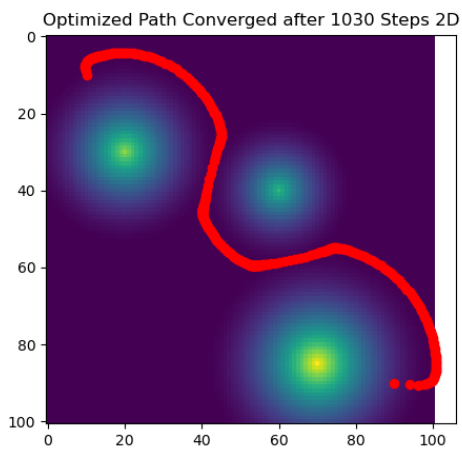


Fig. 12: Optimized Path Converged after 1030 Iterations.

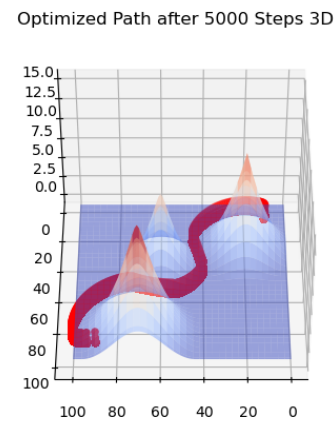
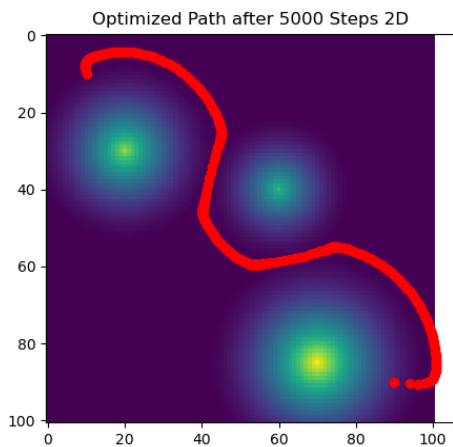


Fig. 13: Optimized Path after 5000 Iterations.

**Better because it respects both start and end point smoothness conditions.**

- (d) What will happen if you were to start the procedure from different initial paths?  
Will the final answer be the same every time? Why?

**Solution:**

Will not be the same every time because the optimization is a local optimization, and the final answer depends on the initialization.

- (e) Suppose that the final solution path found by gradient descent, given some initial path and given our original cost function, is not feasible for the robot to execute (it still passes through some high cost regions that you would rather not have the robot pass through)? What simple common-sense strategy can you come up with to try to mitigate this problem? (*No need to write code unless necessary. We expect a very short description of the strategy only.*)

**Solution:**

Well, this is exactly what I was asking on piazza (the question was set to private ;)).

Smoothness to human eyes is different from smoothness to robotic control, it at least involves 2 aspects:

- i. After obtaining the path represented as discrete points, we must do what was done in the last question of homework 2, i.e., parametrize the path to yield continuous functions  $f_x(t)$ . e.g., in quadrotor control we have minimum snap/jerk trajectory.
- ii. The optimization of path should ideally assume continuous path representation, otherwise the path points could end up in pretty bad places, e.g., 2 originally adjacent points going into 2 different abyss, with an un-surpassable hill in between. This can happen as a result of discretization aliasing, when you have non-matching discretization granularity for the path and the obstacle cost map. (map is dense, but path points are sparse) This would involve "curve-minimization", but that's probably non-trivial.

Please submit code for parts (a), (b), and (c), but none is required for (d) and (e). Please provide answers for all parts in your pdf, as usual. In particular, replicate any code snippets in your pdf that you think the TAs will need for grading.